

70 SHEELAM MYTHRI

thesis_report_organized

 set

Document Details

Submission ID

trn:oid:::3618:137318821

Submission Date

May 1, 2026, 2:11 PM GMT+5:30

Download Date

May 1, 2026, 2:21 PM GMT+5:30

File Name

thesis_report_organized.docx

File Size

633.1 KB

63 Pages**19,760 Words****106,048 Characters**

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



CHAPTER 1

INTRODUCTION

1.1 Background of the Problem

The Internet of Things or Internet of Things has changed fast in the last few years. It has completely changed the way people interact with their homes. Smart homes, which used to be something you would see in science fiction movies are now very common. They have lots of devices that are all connected to each other like light bulbs, temperature sensors, motion detectors, security cameras, smart locks and voice-controlled assistants. These devices all work together talking to each other over the home network to make life easier and more convenient for people. They also let people control things remotely [27][29].

More and more people use the Internet of Things smart homes are becoming more and more connected and that is making life better for people.. It is also making it easier for hackers to get in and cause trouble. The problem is that the devices that make up the Internet of Things are not made with security in mind. They are made to be cheap and to use power, which is good but it also means they are not very good at keeping hackers out. Most Internet of Things devices do not have powerful processors and they do not have much memory so they cannot run very good security software. That means they are targets for hackers who want to get into home networks intercept communications or use the devices to launch bigger attacks [5][15].

There are kinds of cyber threats that can attack smart homes that use the Internet of Things. One kind is called a Denial-of-Service attack, which's when a hacker sends so much traffic to a device that it cannot handle it and the device stops working. Another kind is called a Distributed Denial-of-Service attack, which's when a hacker uses a bunch of devices that they have already taken over to launch an attack. There are also force attacks, which are when a hacker tries lots of different passwords to try to get into a device.. There are Man-in-the-Middle attacks, which are when a hacker intercepts communications between two devices and maybe even changes them. Each of these kinds of attacks takes advantage of weaknesses in the Internet of Things and they can all have serious consequences for peoples privacy and safety[32].

The Internet of Things devices also make a lot of kinds of data which makes it hard to keep track of everything and make sure it is all secure. The old ways of detecting intruders, which look for patterns of known attacks do not work well in Internet of Things environments. That is because new kinds of attacks are being invented all the time and the old systems cannot keep up. These old systems need to be updated all the time. They often give false alarms, which can be annoying. They also use up a lot of resources, which's a problem for devices that do not have very much power [6][28].

Because of these problems researchers have started using machine learning to make Internet of Things security systems that can adapt to kinds of attacks. Machine learning systems can look at network traffic data. Figure out what is normal behavior. Then they can detect when something is not normal. They can detect both kinds of attacks that they have seen before and kinds of attacks that they have never seen before. Some techniques, like the Local Outlier Factor and Random Forest have been shown to work well in experiments.

This project is building on that work by making an Internet of Things security system that uses machine learning. It is going to use the Local Outlier Factor to detect anomalies Random Forest to classify kinds of attacks and it is going to send email alerts when it detects something suspicious. All of these things are going to be combined into one system that works together smoothly.

1.2 Need and Relevance

The importance of security of smart homes has never been greater. Experts have said that the number of devices in the world is growing at an alarming rate. Devices are already in use in billions and are likely to increase in the future. This includes smart homes that have gadgets such as TVs, lights and health monitors. The vast quantity of devices poses security threats. The existing solutions are inadequate to contain these risks. Smart houses have appliances, both entertainment and security cameras. All these gadgets pose a risk of breaches of security. We require security to safeguard smart homes. The IoT devices are increasing rapidly. So are the security risks. IoT security is important, in the home.

A large factor that makes IoT devices need security is that cyberattacks are ever-evolving. Hackers are quite proficient at locating vulnerabilities in devices and attack numerous devices simultaneously through automated software and botnets. In case of hacking of a smart home system, it may lead to such inconveniences as service disruption or even severe consequences such as loss of money or even lives in case some security devices or even medical equipment are targeted. We require a reliable security system that can be able to identify and act on these threats fast. The reason why IoT security is important is that it can be used to safeguard our devices and personal information in case of cyber attacks. An effective IoT security system must be capable of reacting to the threats promptly to allow us to avert severe issues.

The proposed project is significant as it attempts to provide solutions to problems in a straightforward manner. It is based on Internet of Things microcontrollers such as ESP32 and smart bulbs. The project investigates the behavior of the network when it is functioning as normal and when it is under attack. This implies that the outcomes of the projects are grounded on what occurs, not only on ideas. Machine learning models that are trained on Internet of Things traffic are also used in the project. This renders the system more helpful in life compared to other approaches which only make use of fabricated data. The Internet of things is a feasible project. This is what makes it so handy. This project includes the Internet of Things hardware.

The project is trying to fix a problem in the current research. This problem is that we do not have systems that can find anomalies and then tell us what kind of attack it is and alert us. Most security solutions for Internet of Things devices only do one thing. They. Find anomalies without saying what they are or they use one type of model that cannot handle known and unknown threats. The project is using an approach that combines two methods. It uses a method called Local Outlier Factor to find anomalies without being taught what to look for. It also uses a method called Random Forest to classify the anomalies after they are found. This makes the security solution better. The project is using this approach to make Internet of Things devices more secure, by finding anomalies and classifying them and then sending out alerts.

The importance of this work goes beyond what it means for schools and universities. As more people get home technology like older people and people who are disabled it

becomes really important that these homes are safe. This is also true for people who live in places that're not as developed. A security system like the one we are talking about here can help make sure everyone has a smart home. This system is good because it does not need a lot of power to work so it can be used in different places. It can help people feel safe in their homes even if they do not have a lot of money or access to complicated technology. Smart home security is very important. This system can help make it available, to everyone.

1.3 Problem Statement

Smart home devices are really handy. Can do a lot for us but they also make it hard to keep our homes secure. The things that make up our home systems are not very good at keeping bad people from getting in because they do not have many security features and it is easy to get to them. They have problems, like people using the same login information that the device came with sending information without encrypting it and not being able to update the devices software. This makes it easy for bad people to take advantage of our home devices. The smart home devices are always talking to each other. They make a lot of data so it is hard to figure out if someone is trying to do something bad and stop them. Smart home devices and smart home devices are making it really tough to keep our homes and smart home devices safe.

The security systems we have today do not do a job of keeping our smart home internet devices safe. They cannot find threats or new attacks that are just starting to happen. Some systems can tell when something weird is going on. They do not know what kind of threat it is. Other systems are good, at finding threats they have seen before. They cannot find new threats that they have never seen. Smart home IoT networks need protection. Most security solutions do not send out alerts away or do anything to stop the threat automatically which makes them not very useful when we actually use them. Smart home IoT networks and security systems need to be improved to keep our homes safe.

The problem that this project is trying to solve is that we need a security system for the internet of things. This system has to be able to do a things at the same time. It has to be able to find behavior that's not normal and figure out what kind of cyberattack it is.

It also has to be able to do all of this without using much computer power because the devices on the internet of things do not have a lot of power.

The internet of things security system has to be able to tell us about threats, in time. The internet of things security system must be able to handle all sorts of traffic conditions. It has to be able to do this without making many mistakes. The internet of things security system has to be able to work in smart home networks and it has to be able to handle a lot of devices.

1.4 Objectives

The main objectives of this project are:

- To design and implement a real smart home IoT environment using IoT devices such as ESP32 and a smart bulb communicating over Wi-Fi.
- To capture and analyze normal network traffic generated by IoT devices using monitoring tools such as Wireshark.
- To simulate different network-based attacks on the IoT communication and collect corresponding attack traffic data.
- To preprocess and analyze both normal and attack traffic datasets for behavioral pattern identification.
- To develop and evaluate an anomaly-based intrusion detection system using machine learning models to distinguish normal and malicious IoT traffic.
- To extend the system by integrating automated response and alert mechanisms for real-time IoT threat mitigation.

1.5 Scope of Work

The project focuses on designing and developing internet of things (IoT) smart home security system. It involves the collection of the network traffic of the smart bulbs and ESP32 modules under normal and attack mode. The data obtained is processed with preprocessing (such as data cleaning, encoding of categorical features, feature scaling and dimensionality reduction) and ensures consistency and an improved model. Based on this processed information, machine learning models are developed to identify

anomalies and classify attacks, which means that the system will identify abnormal network traffic and the categorization of the different types of cyberattacks.

Implementation of these models within a single detection system capable of analyzing the input information and providing warnings in case of suspect activity detection is also part of the project. This will enable identification of any threats in the network promptly. But only to a controlled experimental setting with a series of simulated attack scenarios. The platform is not faithful to the large-scale conversations and intricate industrial IoT administrations and extra validation would be necessary to appreciate such execution.

1.6 Engineering, Environmental and Sustainability Considerations

This section examines the Internet of Things security system proposed. What we'd like to know is how challenging it is to construct just how many it costs, what it does to the environment whether it is sustainable and whether it is good, to society. These are very important when we are doing work on engineering projects. They impact the technical component of design, yet also on what we encounter when we come to the actual use of the system. All these ways have to be considered in the Internet of Things security system.

1.6.1 Engineering Complexity

The suggested system is a highly technical engineering endeavor. There are many components, such as the engineering of the IoT hardware and network traffic analysis and data science and machine learning model and software system integration. Design must be designed in such a way that all the components collaborate effectively even parts which prepare the data detect things classify things and send out the alert. We have certain challenges such as ensuring that the features are the same when we are training and testing that offer an excessive amount of data and processing all in real time. All major components of the system include the hardware engineering and network traffic analysis/data science and machine learning model development, and software systems integration. The mechanism is not readily fabricated. A final year undergraduate student can do it using tools and frameworks which are readily accessible.

Table 1: Engineering and Sustainability Complexity Assessment

Dimension	Assessment	Description
Engineering Complexity	Moderate	Multi-module ML pipeline, real hardware, IoT traffic analysis
Cost Consideration	Low-Moderate	Open-source tools, affordable ESP32 hardware, free Python libraries
Environmental Impact	Minimal	Software-centric, low energy hardware, no physical infrastructure
Sustainability	High	Reusable pipeline, adaptable to future attacks, modular design
Societal Relevance	High	Improves home security, applicable to elderly/disabled users

1.6.2 Cost Considerations

A lot of open-source software is used in the project. Free Python libraries. This actually assists in lowers the costs of developing the project. The key components of the underlying hardware are such as ESP32 modules and smart bulb systems. These are machines that can be purchased by anyone without the need to spend a considerable amount of money. Computers are used to train the machine learning models. We do not require computers, and equipped with excellent graphics cards to do this. The entire system is extremely inexpensive to develop. This implies that it can be easily copied by other researchers and individuals who may work with the systems without necessarily having much money to add to its contents. In case one wishes to use this system in areas where he or she will require to pay such as hosting the alert notification system and maintaining the other aspects of the system in excellent working conditions. The project utilizes open source software. Free Python software to save on costs.

1.6.3 Environmental Impact

The proposed system has a minimal impact on the environment. In this project, nothing is set up that requires a lot of hardware or the creation of infrastructure that consumes a lot of energy. The software components are designed to run on standard computers with affordable computing capabilities. Our machine learning models, such as LOF and Random Forest, as opposed to complex deep learning algorithms that consume a lot of energy, contribute to a smaller carbon footprint of the systems. Nor do we have servers at all, but rather make use of smart Python packages, a factor that contributes to the overall low usage of energy both in the erection of the system and in the operation of the system.

1.6.4 Sustainability Aspects

The system is designed in a way to be long-lasting. It is constructed in such a manner that allows us to upgrade or change components without tying up the object. This implies that the system can evolve, with the introduction of Internet of Things technologies and attack patterns. We can update the pipeline and model artifacts provided by the preprocessing with information to maintain them up-to-date. Working with free frameworks implies that we are not dependent on tools such as those produced by a single company. This makes the system less likely to be outdated. The mechanism employs a mixture of two types of learning supervised learning to discover threats. This aids in making the system remain effective. Makes it stronger against of old and new threat. The system remains efficient, during its operating lifetime.

1.6.5 Societal Relevance

The implications of the proposed system on the society are truly enormous. Increasingly more people are employing domestic systems even the aged, those who are unable to go about and people with young children. These individuals are dependent on systems which are remotely controllable. When a person has broke into these systems it can be extremely detrimental to the occupants. It achieves this through offering a security solution that is simple to operate and works effectively, on devices that are not that powerful. It implies that people can have a safer smart home technology. This is significant since there are bad individuals who would attempt to use the internet to create issues and commit crimes. Numerous nations are endeavoring to prevent this. The given system is one of these attempts.

1.7 Organization of the Report

The project report chapter is broken into six chapters. It contains a reference list and list of appendices as well. This is to give a thorough picture of the project between initial and final.

The introduction is the 1st chapter. It provides the background and the background of the project. The project concerns home internet facilities and the security issue that it is encircled by. The problem has also been defined in this chapter. States the objectives of the project. It discusses considerations of the engineering and sustainability. Here the outline of the report structure is described as well.

Chapter 2 is the review of what other individuals have studied in regards to the IoT security. In this chapter, they contrast the techniques of detection and explore what is lacking in the research up to date.

The approach the project adopted is the topic of the 3rd chapter. It begins with what was learnt in the stage and how the problem statement and objectives have been enhanced. The reasons why a hybrid approach was used in this chapter are explained. The costs and whether it is sustainable and whether it is possible are also taken into account.

The 4th chapter includes the discussion of the development and implementation of the project. It examines its origin and that of the various components. It also examines the way the what was put together and what were the obstacles. What was learnt in the implementation is also discussed in this chapter.

The Chapter 5 is on the project outcomes. It examines the manner in which the system was functioning. This chapter Interprets the results. It makes sure the goals of the project have been achieved, and test the system against standard methods of doing things.

The 6th and last chapter is a recap of what has been learnt under the project. It discusses what has not been effective and what it can do better in future. This chapter also examines the ways in which the project could be extended and improved. The home IoT environment and the security challenges are discussed in the project report. These chapters are included in the report to give an image of the project.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview of Existing Work

The Internet of Things devices are becoming really popular. This has led to a lot of research, on Internet of Things security. People are trying to find ways to stop things from happening to Internet of Things devices. They are working on making systems that can detect when something is wrong and figure out what the problem is. Some people are building test areas to try out their ideas. Others are making collections of data to help train computers to spot problems. They are using computers to look at Internet of Things network traffic and find things that do not seem right.

Some of the oldest and most basic contributions to the field are the ones concerning the design and construction of IoT security testbeds. The study by Alrawi et al. [1] outlined a model of the automated evaluation of an IoT device security vulnerability under realistic conditions of home network. This system allowed the identification of the typical vulnerabilities in a systematic manner, including the unencrypted communications, weak authentication, and vulnerable network services. Likewise, Siboni et al. [2] introduced an IoT security testbed that was aimed at helping to test vigorously intrusion detection and protection systems within a controlled experimental environment. Both these contributions emphasized the necessity of empirical assessment in the realistic context, as well as showing how hard it is to create the complete complexity of actual IoT traffic patterns in the laboratory.

Simultaneously, a lot of effort has been put into the creation of representative datasets of IoT traffic. The YourThings dataset, an extensive annotated dataset of smart home device traffic, was generated by Sivanathan et al. [4] and has been a popular benchmark in later intrusion detection studies. The dataset includes traffic of a wide range of smart home devices in various conditions of application, which are labeled and could be used both in a supervised learning strategy and unsupervised style. More recently, Kim et al. [14] proposed CICIOT2023, a massive real-time benchmark that is specifically created to be used in the assessment of IoT attack detection systems of a wide variety of types of attacks. Although these datasets are important community resources, they are

inevitably fixed and might not fully reflect the temporal dynamics and device heterogeneity of real deployment setting.

Specific studies have also focused on the characterization of real-world IoT traffic behavior. The research by Apruzzese et al. [6] on the traffic patterns of smart home IoT revealed that each IoT device shows unique signatures that could be utilized to distinguish the devices and identify anomalies. This article has shown that IoT devices are predictable yet sometimes irregular in their communication patterns and that these patterns differ greatly among devices and usage scenarios. The results of this study indicate the significance of the baseline modeling of devices to detect anomalies successfully in smart homes.

There is significant interest in machine learning solutions to IoT intrusion detection. Network intrusion detection with random Forest, Support Vector Machines (SVM), Decision Trees and k-Nearest Neighbors have been utilized with encouraging outcomes. Mitchell and Chen [16] conducted a survey of intrusion detection in internet of things and cyberphysical systems and found out that machine learning techniques have much greater benefits than traditional rule-based techniques in regard to flexibility and scope of detection. Hodo et al. [5] proved the usefulness of neural network methods of DoS and DDoS attacks detection in IoT infrastructures, which were highly accurate on artificial traffic data. Doshi et al. [15] compared machine learning models to detect DDoS attacks in IoT networks in particular, and the authors discovered that ensemble classifiers like Random Forest performed better and more consistently than single classifiers.

IoT security has received unsupervised anomaly detection techniques, such as the Local Outlier Factor (LOF) algorithm by Breunig et al. [10], clustering algorithms, and Isolation Forest [11], aiming to detect threats that are not known before. These strategies do not need labeled attack data training, and are especially useful in settings where the set of possible attack types is not completely known a priori. They are however vulnerable to higher false positive rates especially in a setting where normal network behavior is very variable or is influenced by time. Patcha and Park [24] gave a comprehensive survey of methods of anomaly detection in network security and they were able to pinpoint the strengths and inherent limitations of statistical and density-based methods.

Autoencoders, long short-term memory (LSTM) networks, and convolutional neural networks (CNNs) have been used to detect intrusion in IoT with promising outcomes on benchmark datasets that use deep learning techniques. Javaid et al. [17] and Abeshu et al. [20] have shown that deep learning models are capable of attaining high detection rates on high-dimensional, complex network traffic data through the automatic learning of features that are relevant. Nevertheless, they are costly in terms of training data and computing resources and are therefore not as applicable to implementation on resource-constrained IoT gadgets. Moreover, deep learning models often provide less interpretability than other machine learning methods, which may be a drawback in security-vital systems where it matters to have knowledge about the underlying algorithm that informs a detection decision.

Recent literature development shows that there is an increasing acknowledgment of the benefits of hybrid solutions, combining various detection strategies. Reviewing applications of deep learning in IoT cybersecurity, Ferrag et al. [35] reported that unsupervised anomaly detection with supervised classification provided a better coverage of the detection. Ring et al. [36] conducted a survey of network-based intrusion detection systems and suggested multi-layer architecture as the most appropriate to counter the extent of threats experienced in the current network settings. These suggestions have a solid theoretical and empirical basis of the hybrid approach embraced in the proposed project.

2.2 Comparison of Methods / Approaches

The existing approaches to detecting intrusions in the IoT network and classifying threats can be roughly divided into four different paradigms: signature-based, anomaly-based, machine learning-based, and deep learning-based paradigms. All these paradigms are characterized by specific strengths, weaknesses and appropriatenesses to various deployment situations. These trade-offs need to be understood to make the best possible choice on the best approach to use in the proposed smart home IoT security system.

The oldest method of network intrusion detection is signature based detection systems. Such systems have a database of recognized attack signatures, patterns of network behavior linked to a particular attack type. The traffic coming in is matched to these

signatures and an alert is raised. Although signature-based systems are very accurate and low false positive rates on known attacks because of the specificity of the matching process, they inherently cannot identify unknown, new, or changing attack patterns, which will not be matched in the signature database. The need to periodically update databases and the increasing complexity of attack signatures are also high maintenance overheads. Organizational security standards and guidelines, like those of NIST [32] clearly acknowledge these limitations, and suggest additional detection strategies in the environment where the threat is changing.

Anomaly-based detection systems is a radically different approach whereby the normal behavior of the network is modeled and anything that falls out of this model is considered an anomaly. Local Outlier Factor (LOF), Isolation Forest, statistical thresholding, and clustering are some of the techniques employed to define normal traffic and identify outliers. The key benefit of anomaly-based systems is that they can identify attack patterns that have not been seen before, such as zero-day threats, without having labeled attack data. Nevertheless, there are high false positive rates associated with these systems, especially in dynamic environments where the normal traffic behaviour varies with time or is naturally variable. The detection accuracy heavily depends on the quality of the normal baseline model, and ill-calibrated models can treat many normal and unusual but legitimate behaviors as threats.

The classification methods are machine learning-based, which are supervised learning algorithms that have been trained on labeled data and are used to classify network traffic into normal or a particular type of attack. Random Forest and SVM algorithms, Decision Trees, and k-NN have shown great performance in the benchmarks of IoT intrusion detection. These methods have a high classification accuracy on known types of attacks, efficient inference and relative resistance to noisy and high dimensional data. Their major drawback is that they require labeled training data, which might be hard to access in adequate amounts and diversity, especially with less frequent types of attacks. Selection of features and quality of data is also sensitive to performance.

Deep learning methods are extensions to supervised classification that involve neural network structures that learn feature representations on raw or minimally processed data. Although these methods have the potential to reach state-of-the-art accuracy on challenging classification problems, they need enormous amounts of training data,

extensive amounts of computation to train, and specialized hardware to effectively infer. This renders deep learning infeasible due to the insufficient resources of IoT devices with no inference accelerators on resource-limited platforms. Moreover, deep neural networks can be black-box, which makes them less interpretable, which is an issue in applications where decision transparency is important (e.g., security-related).

Table 2: Comparison of IoT Intrusion Detection Methods

Method Type	Advantages	Limitations	IoT Suitability
Signature-Based	High accuracy for known attacks; low false positives	Cannot detect unknown or zero-day attacks; requires constant updates	Limited; cannot handle evolving IoT threats
Anomaly-Based (LOF, IF)	Detects unknown attacks; no labeled attack data required	Higher false positives; sensitive to baseline variability	Good for unknown threats; needs careful parameterization
ML-Based (RF, SVM)	High classification accuracy; handles structured data well	Requires labeled data; performance depends on data quality	Well-suited; computationally efficient
Deep Learning (AE, LSTM)	Automatic feature learning; effective on high-dimensional data	High compute/data requirements; low interpretability	Limited for resource-constrained devices
Hybrid (LOF + RF)	Combines unknown and known attack detection; reduced false positives	More complex design; requires both labeled and unlabeled data	Optimal for smart home IoT deployment

The current trends of research indicate that none of the existing approaches is adequate to handle all the IoT security challenges. It has been observed that hybrid methods involving a combination of anomaly detection and classification methods have performed better. As an example, anomaly detection might be employed as a first line of defense in order to detect suspicious activity, and classification models can detect the exact type of attack. This multi-level method will diminish the false positive and increase the accuracy of detection.

According to this analogy, machine learning-based techniques in conjunction with anomaly detection offer a viable and effective solution to IoT security. They provide compromise between real-time smart home environments in terms of accuracy, adaptability, and computational efficiency. This is the point, on the basis of which the approach in the current project was taken.

2.3 Critical Analysis of Literature

What the critical analysis of the literature available in the field of IoT security reveals are that there are still some gaps that are widespread and limitations to the usefulness and practicality of existing solutions. Despite the significant progress that the theoretical framework has made and experimental authentication of the IoT intrusion detection systems has been realized, there is a trail of overlapping gaps that dominate the literature and which have not been filled satisfactorily.

The lack of a unified detection frameworks is one of the largest and most widespread limitations of the available literature. The vast majority of the literature assumes the use of anomaly detection and attack classification are two different and independent tasks and creates them and evaluates them separately. This dichotomy is then unsatisfactory because anomaly detection alone could not identify the nature of a detected threat and classification models did not have the capability of extrapolating to new patterns of attack. The practical implication of this is that neither of the two kinds of systems is fitted to the broad scope of security and the area of commerce between them to form a combined system has received limited research.

The second critical limitation is the overuse of fixed, benchmark datasets to work out and to evaluate the models. Though databases such as YourThings [4] and CICIOT2023 [14] can provide convenient standardized reference points, they are not always able to reflect the heterogeneity of the devices, time dynamics, and scale of the real world IoT deployments. It is always not guaranteed that the findings of studies based on such data sets can be generalized to manufacturing environments, and Apruzzese et al. [6] have found that in IoT real-life traffic conditions, which are highly place- and device-specific, the findings may be quite different. This brings about the use of real hardware-based information-gathering, as desired in the proposed venture.

The problems associated with the quality of data, like the dis-proportion of classes, excessive dimensionality, overlapping features also represent the areas of concern in the literature. The excess normal traffic over attack traffic is also often larger in most types of attack, resulting in most IoT datasets being drastically class-imbalanced. Trained models that do not reduce this imbalance will be poor at polymorphic minority attack classes, reducing their utility in practice. Redundancy of the features and collinearity escalate the computational cost and may worsen the performance of the model with noisy features. The literature [16][24] has described the importance of stringent preprocessing and feature engineering in surmounting these challenges but is not consistently implemented with published solutions.

Another significant practical limitation that is not adequately addressed in many studies of deep learning research is the feasibility of running computer-intensive deep learning models on the resource-limited IoT devices. Deep learning approaches can achieve high accuracy on benchmark tasks, but generally have more resource requirements that are infeasible on the hardware of most IoT devices. This creates a gap between what is measured by academic performance and what can be actually deployed to practice that makes much of what we publish useless.

False positive rates in anomaly detection systems are very high and this is yet another important issue. The essence of anomaly-based detection is that it will probably flag more legitimate yet suspicious traffic (when it is malicious) particularly in dynamic IoT systems where the behavior of the device will change depending on the usage history, software, or network status. False alarms are damaging to user trust, leading to alert fatigue and can cause security staff to ignore the factual dangers. The solutions that exist tend to fail to systematically regard false positive reduction as an aim of design, thereby resulting in systems that may be impractical to manifest in reality.

Real-time detection and automated response is a serious limitation of operations since most published systems lack this feature. Traffic analysis and post hoc assessment have an acceptable coverage on literature but full systems capable of processing inbound traffic, detecting threats, and dispatching alerts within time scales that can be operationally relevant are not common place. This disconnection limits the application of the existing research about the functioning of the IoT security in reality. Other challenges are scalability and flexibility. Their models have been validated in a number

of studies under controlled conditions such as IoT testbeds [1][2], which are not reflective of large scale or real world applications. Perhaps the systems would not work as well under uncontrolled conditions where the systems are exposed to diverse devices, varying network conditions and dynamic patterns of attacks.

Overall, the critical review of the existing literature indicates that the solutions provided are helpful in providing insights and partial resolutions, yet fail to provide an overall, scalable and real-time IoT security system. These drawbacks show that it needs an all-encompassing solution which includes anomaly detection and classification, real-world data use, false positive reduction, and responding and monitoring real time. The proposed work is based on this.

2.4 Research Gaps Identified

Critical analysis of the available literature results in the discovery of multiple particular research gaps that directly spur the design and implementation objectives of the proposed project. These are specific gaps that will reflect tangible points where the existing solutions are failing, and where the suggested hybrid solution will add significant value.

The biggest and most notable one is the lack of the combined hybrid detection structures. Solutions in use today mainly treat either anomaly detection, or attack classification as an isolated problem, and not as part of a single system that can use the complementary strengths of both methods. The proposed project will fill this gap by integrating LOF-based anomaly detection and Random Forest classification in a consistent, end-to-end pipeline.

The second gap is related to the use of static datasets to evaluate the system. Majority of available literature assesses intrusion detection system on fixed, pre-analyzed data which may not capture the variability of behaviors in actual the internet of things. The proposed project produces a dataset since it uses actual ESP32 microcontrollers and devices that are of smart bulbs and record traffic in real operating conditions, which makes the proposed project more similar to a real-world smart home network.

The third gap is the absence of a systematic approach to the issue of class imbalance and preprocessing issues. The consequences of the imbalance of the classes on the model training, as well as the significance of the consistent preprocessing of the pretraining and inference stages, are not fully addressed by many of the existing

systems. These challenges are explicitly addressed in the proposed project by use of oversampling techniques, saved artifacts in its preprocessing and strict pipeline management.

The fourth vulnerability is that most published systems do not have real-time alerting and automated response mechanisms. To address this gap, the proposed project will incorporate an SMTP-based email notification system that will send notifications when threats are detected, making the security framework effective.

The fifth gap relates to scalability as well as the considerations of real-world deployment. Most solutions that are currently available are tested only in very controlled and small-scale testbed settings, and are not taken into account with regard to scaling and the needs of larger or more diverse IoT networks. Although the proposed project is also deployed in the controlled environment, its modular structure and the choice of a lightweight model are specifically intended to support scaling to bigger deployment scenarios in the future.

Finally, the issue of high rates of false positives, especially in anomaly-driven systems is also a big gap. False warning signals may reduce user trust and dependability, particularly when dealing with a dynamic active IoT environment whereby normal states can vary. Nevertheless, the existing solutions often have no tools to address this issue so more accurate and balanced detectors are required.

In order to address the found gaps, the proposed project is directed at the development of a hybrid system of the IoT security on the basis of real traffic of devices. The system incorporates the Local Outlier Factor (LOF) to identify anomalies and the Random Forest to carry out classification to identify the unknown and known attacks. It also features real-time monitoring and alert features, contributing to the pragmatic use. The project will ensure that the system is put into practice in the real-world by the implementation of the actual IoT solutions such as ESP32 and smart bulbs, which will enhance their reliability and efficiency.

Table 3: Summary of Key Research Papers Reviewed

Ref.	Title / Focus	Method Used	Key Finding	Limitation
[1]	Automated IoT Security Testbed Design	Testbed Framework	Identifies common IoT vulnerabilities	Limited to testbed scale
[4]	YourThings: Annotated Smart Home Dataset	Dataset Development	Rich labeled traffic benchmark	Static; not real-time
[6]	Smart Home IoT Traffic in the Wild	Traffic Characterization	Device-specific patterns observed	No IDS component
[9]	Random Forests (Breiman)	Ensemble Classification	High accuracy, robust to noise	Requires labeled data
[10]	LOF: Density-Based Outlier Detection	Unsupervised Anomaly Detection	Effective for unknown attacks	False positives in dynamic environments
[14]	CICIoT2023: Real-Time IoT Dataset	Dataset + Benchmark	Diverse attack types covered	Not device-specific
[16]	ML-Based IDS for IoT Networks	Supervised Classification	ML outperforms signature-based	Limited to known attacks
[35]	Deep Learning for IoT Cybersecurity	Deep Learning	High accuracy on benchmarks	High resource requirements

CHAPTER 3

APPROACH / DESIGN METHODOLOGY

3.1 Overview of Phase-I Work and Outcomes

The stage of exploration and preparation (Phase-I) was the phase of the project that led to the background knowledge, experimental infrastructure and initial technical insights on which the design judgments in Phase-II were informed. The point of Phase-I was to find out how the IoT network traffic works and determine the potential of using machine learning techniques to make security threats detected in the smart home environment.

Phase-1 was experimentally tested using the own commercial IoT hardware, i.e. ESP32 microcontroller modules and a smart bulb device, connected with a common local Wi-Fi network. The ESP32 devices were programmed to periodically send messages on the network that was simulating the usual behavior of an IoT device like heartbeat, status, and control messages. The smart bulb was configured to accept and respond to control signals on the identical network that was typical of one of the consumers in the experiment with IoT.

Traffic on the network was captured with Wireshark which is a freely available, open-source, packet-level packet-analysis tool and could provide a detailed breakdown of packet-by-packet network traffic. Tapes of the traffic were captured in cases when the traffic was functioning normally and when the simulated attack conditions were underway. The simplest DoS flood attacks and network scans were termed as Phase-I attack scenarios. The traffic was exported as formatted CSV files having features according to the packet header data and timing information, including source and destination IP addresses, the protocol type, port numbers, packet length, the time between the packets and packet count.

To check the quality of the data and prepare the data to be subjected to the first analysis, the initial preprocessing of the captured data was performed. This involved identifying and dropping off blank/duplicate columns, imputing missing values using a median,

and placing the categorical variables such as protocol type field and flag fields into numerical values and after which Min-Max normalization was applied to ensure that the ranges of all the features were the same. Exploratory analysis was performed to examine the distribution of features, to examine the correlation and the degree of separation between normal traffic and attack traffic classes..

To test how detectable the attack patterns in the collected traffic data were, preliminary experiments with simple anomaly detection methods, such as simple threshold-based methods, and an initial implementation of Local Outlier Factor were performed. These experiments established that attack traffic has statistically significant deviations of normal baseline behavior on a variety of features, which is evidence that the anomaly detection approach is feasible. Nonetheless, the experiments also established that simple single-feature thresholding could not be used to reliably detect and that multi-dimensional analysis with LOF or analog density-based methods were required to perform satisfactorily.

The main results of Phase-I can be as follows. To begin with, a working experimental IoT was created to offer the infrastructure to be used in the later stages of collecting traffic in Phase-II. Second, a representative sample of the IoT network traffic in the case of normal and basic attacks was gathered and pre-processed. Third, it was established that machine learning-based anomaly detection can be used to detect intrusion on IoT. Fourth, the shortcomings of single-model and single-technique methods were discovered, which made them conclude that a hybrid framework of anomaly detection and attack classification would be required to cover threats in a comprehensive way. These results were directly used in the approach and system design of Phase-II.

3.2 Refinement of Problem and Objectives based on Feedback

After the Phase-I was complete and the preliminary results and feedback were reviewed with the project supervisor, the problem statement and project objectives were further defined in accordance with the practical requirements and technical challenges that occur during the initial phase. These improvements led to a narrower and more detailed

scope of the project which covered both the theoretical and practical aspects of IoT security.

The formulation of the initial problem of Phase-I was centered on the general task of finding abnormal dynamics of IoT network traffic. Though this provided an excellent starting point, the feedback indicated that it was needed to expand the scope as well beyond detection to incorporating the identification of specific kinds of attacks and offering the possibility of responding in real-time. This expanded framing is nearer to system effectiveness concerns, indicative of the practical wants of a useful security system, and can more effectively be employed in addressing serviceability.

The feedback also highlighted the necessity of providing the consistency of the preprocessing between training and inference phases of the pipeline. Previous experiments had demonstrated inconsistency between the transformations of features employed in the training and testing to be a cause of degraded performance. One important technical requirement of Phase-II was to solve this problem by determining the systematic serialization and reuse of preprocessing artifacts.

The objectives were reduced to specifically include the development of an automated alert system, which was not included in the original project scope. This element was incorporated since it was realised that a practical security system cannot work on detection but it should have the capability to inform its operators timely when the threats in detection are being addressed so as to give human operators the latitude to respond properly to the detected threat. It emerged that the email-based SMTP alert system was the most viable and the most available tool to this effect, considering the context of the project.

Additional enhancements were made to the assessment methodology and clear evaluation artifacts like AUC-ROC analysis and confusion matrix visualization were added. These extensions ensure more accurate and detailed measurement of system performance especially the trade-off between false positive rate and sensitivity in detection that is of critical importance in the practical application.

3.3 Selection of Approach / Methodology

The hybrid LOF and Random Forest methodology was chosen due to the systematic analysis of the other approaches, which was based on the literature review findings and results of the Phase-I experiment. Four general methodological choices were taken into account: signature-based detection, standalone anomaly-based detection, standalone supervised classification, and the hybrid combination of anomaly detection and supervised classification.

Signature-based detection was dismissed initially because it lacked the ability to identify types of attack patterns that it had not learned. A system which can detect only previously-categorized attack signatures would be of little practical use in a smart home IoT environment where new variants of attacks are being developed all the time and where the attack surface is itself complex and evolving. The maintenance cost involved in maintaining signature databases up to date is another cost in operation that is not proportionate to value in this regard.

LOF Algorithm Standalone anomaly-based detection was also a candidate method because it is capable of detecting unknown threats without the need to have labeled attack data. LOF quantifies the local density of each data point with respect to its neighbors and finds points with local density that is significantly different than that of its neighbors as anomalies. This solution is especially the most appropriate to a IoT environment where the scope of the possible types of attacks is not known a priori. The weakness of standalone anomaly detection, however, is that standalone anomaly detectors do not determine the nature of a detected threat, they are only able to provide a binary anomaly/normal classification. This restricts the usability of the system output and minimizes its usefulness.

Single supervised classification with the Random Forest algorithm was also an option because it has good performance on marked IoT intrusion detection datasets, it is resistant to noisy and high-dimensional data, and it is computationally efficient as compared to deep learning methods. Random Forest constructs a forest of decision trees, each of which is also trained on a random sample of training samples and features,

and combines their predictions using majority voting. Such an ensemble mechanism is not only highly accurate but also less variable as well as inherently resistant to overfitting. Nonetheless, labelled training data representing every type of attack of interest is still required during supervised classification and it cannot be generalized to novel types of attacks, which is a significant limitation when operating in dynamic IoT environments.

The hybrid methodology of LOF anomaly detection and the Random Forest classification was chosen as the best methodology that would be used in the proposed system because it will overcome the drawbacks of the two standalone methods, and retain their advantages. The LOF model can be used as the first-line detector in this framework to indicate those data points which are far out of the normal baseline traffic pattern. The data points that are considered as anomalous by LOF is then sent to the random forest classifier which classifies them under a particular attack category based on the learned feature patterns. This stratified architecture allows the system to identify known and unknown attacks, offer more specific threat classification to take actionable response on an incident, and is more accurate in general than either of the two parts.

The specific selection of the Random Forest as a classifier of the features during the classification but not the rest of the supervised algorithms such as SVM or gradient boosting were explained by a series of factors. In preliminary tests, Random Forest fared better, and its capability to handle class imbalance and high-dimensional feature spaces gracefully compared to SVM. Its natural features significance rank meant that it was possible to interpretably examine the most differentiating traffic characteristics. It also had good training rates and inference rates compared with gradient boosting training algorithms and was also able to provide near real time processing requirements.

3.4 Technical Approach and Key Elements

The proposed IoT security solution is carried out as a multi-step data processing and analysis pipeline where the network traffic raw data stream is fed into the system sequentially through preprocessing, dimensionality reduction, anomaly detection, attack classification, and alert generation modules. The architecture provides a clean separation of concerns among processing stages, provides uniform data transformation,

and supports the independent development, testing and optimization of each component.

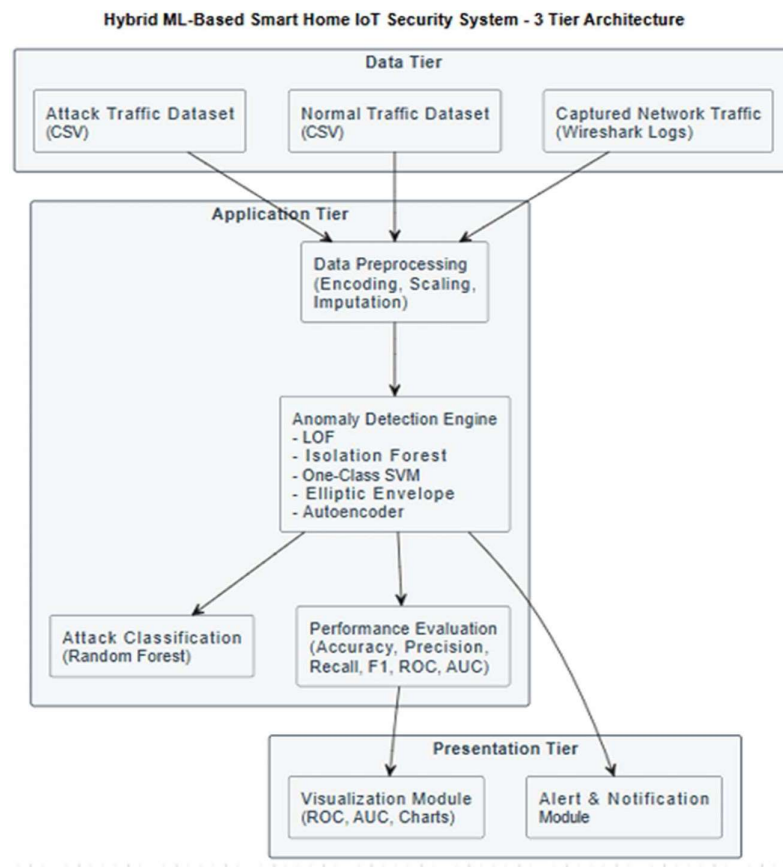


Fig. 1: Hybrid ML-Based Smart Home IoT Security System Architecture

The system architecture shows how raw network traffic goes through steps. It starts with preprocessing. Then it uses Local Outlier Factor (LOF) to detect anomalies in the traffic. After that it uses a Random Forest model to classify the traffic. Finally it generates alerts based on the classification results. The raw network traffic is processed using LOF for anomaly detection and Random Forest for classification to generate alerts. The architecture explains how the traffic flows through these steps to identify issues.

Data Collection Module:

The data collection module will collect raw network traffic in the IoT experimental environment. The environment comprises an ESP32 microcontroller and a smart bulb device that work on the same Wi-Fi network. Wireshark is used to capture traffic and

logs all packets that pass across the network interface on a packet-by-packet basis. Traffic is also captured as CSV with such features as source and destination IP addresses, source and destination ports, the type of protocol, packet length, TCP flag values, inter-arrival time, and flow-level statistics, including the number of packets and the number of bytes. The traffic data are gathered under usual operating conditions and during the implementation of simulated attack scenarios to make sure that both classes are represented in the training set.

Data Preprocessing Module:

Preprocessing module converts raw traffic data into an analysis-ready and consistent format. The following sequential steps make up the preprocessing pipeline: delete irrelevant and duplicate columns that do not contribute to predictive power; detect and impute missing values through median imputation to maintain data distribution; ordinal and one-hot encoding of categorical features (protocol type, flag fields) where appropriate; and normalization of all numerical features with standard scaling (z-score normalization), to assure the similarity of the magnitude of features. All preprocessing transformations are trained on the training data and are easily serializable to apply to test and inference data.

Dimensionality Reduction Module:

Principal Component Analysis (PCA) is used to create a lower dimensionality of the processed feature space. PCA determines the directions of maximum variance in the training data and projects all data onto fewer principal components which jointly capture most of the explained variance. This step removes redundant and correlated features, downstream model training and inference requires less computation, and the danger of overfitting that can be caused by high-dimensional feature spaces is minimized. The PCA transformation is trained on the training data and saved as a serial and the preprocessing artifacts are saved to be used consistently during inference.

Table 4: System Component Summary

Component	Method / Tool	Function
Data Collection	Wireshark, ESP32, Smart Bulb	Captures real network traffic under normal and attack conditions
Preprocessing	Pandas, NumPy, Scikit-learn	Cleans, encodes, scales, and normalizes raw traffic features
Dimensionality Reduction	PCA (Scikit-learn)	Reduces feature space while retaining maximum variance
Anomaly Detection	Local Outlier Factor (LOF)	Identifies deviations from normal traffic baseline
Attack Classification	Random Forest (Scikit-learn)	Classifies anomalies into specific attack categories
Alert System	SMTP (Python smtplib)	Dispatches email notifications upon threat detection

Anomaly Detection Module:

The anomaly detection unit uses the Local Outlier Factor (LOF) algorithm to detect a data point that is significantly different than the normal network traffic baseline. LOF directly calculates the density of a data point, relative to that of its nearest neighbors, to obtain a local reachability density score. The points whose local density is much lower than that of their neighbours are given a high LOF score and are considered to be anomalies. The LOF model is only trained using normal traffic data and this guarantees the model learns a clean model of normal IoT network behavior. The contamination parameter is used to regulate the ratio of samples in the training set that are supposed to be anomalous, and is tuned by domain knowledge and empirical validation.

Attack Classification Module:

The attack classification module uses a Random Forest classifier that is trained on attack data with labeled attacks to classify the identified anomalies based on each type

of attack. A dataset of labeled samples of various attack categories such as DoS, DDoS, brute force and MITM attacks is used to train the classifier. The imbalance of classes in the training material is resolved using the Random Over-Sampler method which creates artificial samples of the underexpressed classes in order to balance the distribution of the classes within the training data. Random Forest hyperparameters such as number of estimators, maximum depth and minimum number of samples per leaf are optimized using cross-validation to maximize classification performance. The trained model will provide a predicted label of the attack type, and a confidence score of each labeled anomaly.

Alert System:

The alert generation module will receive the results of the classification module, and will email messages whenever an attack is detected. The module uses benefits of the Python smtplib library to connect to an SMTP email server and deliver messages using a formatted notification format with the type of attack detected, confidence score and time. By providing appropriate and timely notifications that can be responded to to avoid delays in the working system by the human operators, the module ensures the system becomes more useful in its functioning, thus enabling the operators to take appropriate response to the incident.

A single pipeline is formed which integrates all these elements where the data enters the collection, preprocessing, detection, and classification up to the alert generation. This procedural approach will ensure proper identification, reduction of false positives and proper functionality of the system. This system will also be scalable to be added to later as it is designed to give flexibility to the changing requirements of the IoT security because of its modular design.

3.5 Supporting Analysis / Design Considerations

The proposed IoT security system has been developed keeping in mind many technical and working factors that influence the functionality of the system, its durability and its viability in operation. All of these factors shaped some of the design decisions made at all levels of the pipeline and contributed to the overall stability and efficiency of the system.

Scalability was also taken care of by transitioning to a modular pipeline architecture that decouples data collection, preprocessing, detection, classification and alerting into autonomous units. The design enables the optimization, scaling or replacement of individual modules without compromising the functionality of other components. The preprocessing and model inference phases are performed using efficient data structures and vectorized operations, so that the system can process more network traffic without having to increase processing latency in corresponding proportions.

Design constraints were also majorly concerned with computational efficiency considering the resource constraints of IoT devices and real-time processing needs of a network security system. This constraint is evident in the choice of LOF and Random Forest instead of more computationally intensive methods like deep neural networks. Dimensionality reduction with PCA also lowers the computational cost of downstream model operations by making the feature vectors to which each model operates smaller.

Class imbalance was declared a major challenge to consider in Phases-I experimentation. The network traffic dataset has a high degree of class imbalance, with normal traffic samples considerably outnumbering those of each type of attack. Without control, such an imbalance would lead to the Random Forest classifier developing a systematic bias of predicting the most common class leading to low recall of rare but significant attack types. Random Over-Sampler method was used on the training data to re-establish the classes proportion and weighting of classes was also performed during training of the model to further penalize misclassification of minority classes.

It was found that preprocessing consistency between training and inference stages was a key factor in the consistent behaviour of a system. In Phase-I, it was found that discrepancies between the transformation of features used in training and in testing led to serious performance deterioration. To deal with this, all the preprocessing transformations, such as imputation parameters, scaling statistics, and PCA projection matrices, were serialized with the joblib library of Python, after being fit to training data. These stored artifacts are loaded and used uniformly at the inference stage where test and production data are subjected to the same transformations that were used at training.

One particular design goal was to minimize false positive reduction, driven by the fact that in a system with high false alarms, the utility of the system decreases and a lack of trust develops in the user. The parameter of LOF contamination and neighbourhood size were adjusted empirically to strike a balance of sensitivity and specificity of detection. The layered architecture, where anomalies identified by LOF are further filtered by a second-stage classification by Random Forest, gives further filtering and allows fewer false positive alerts, as only those samples that not only seem to be anomalous but also are classified by the Random Forest model as a known attack type will trigger an alert.

3.6 Tools / Techniques Used

The implementation of proposed IoT security system is premised on an appropriately selected list of software tools, hardware, and machine learning techniques. These are elaborated in the following with justification as to why they were selected.

Programming Language:

The primary Python programming language (version 3.x) is used in all the data processing, model development and integration. The huge number of data science and machine learning libraries, its syntax of building pipelines and its popularity with the research community make python the natural selection in this project.

Libraries and Frameworks:

Some libraries in Python aid in processing, model development, and visualisation of data. Data preprocessing is always performed using Pandas and NumPy with the help of which missing data are managed, data transformations are made, and arithmetic operations are performed. Scikit-learn is a library that implements machine learning algorithms, such as anomaly detection and classification models, and assessment metrics. Matplotlib is used to display performance on various models, and generate graphs, and works with confusion matrices and more.

Machine Learning Techniques:

The system builds upon different machine learning methods to enhance detection and classification. Local Outlier Factor (LOF) identifies anomalies by identifying

deviations to normal behavior in the networks without the requirement of labeled data thus applicable in the detection of unknown attacks. Random Forest is designed to classify multi-class attacks and has high accuracy, robustness and is able to handle complex data. Principal Component Analysis (PCA) streams out dimensions that redundantly capture features with an aim of enhancing computational efficiency and model performance.

Network Surveillance:

Wireshark logs and dissects network traffic of IoT devices. It enables a close analysis of packet data, such as protocol contents, packet size as well as communication pattern. This is a tool necessary to monitor network traffic (realistic) during normal or attack perpetrated conditions.

Hardware Components:

ESP32 modules and smart bulb systems are IoT devices that can be used to form a controlled test environment. These devices cause actual traffic on the network, which helps to replicate patterns of real communication and evaluate the system in real-life circumstances.

Communication Protocol:

MQTT (Message Queuing Telemetry Transport) is the communication protocol used by the system between the devices in the IoT. MQTT is fully lightweight, efficient and configured to low bandwidth environments hence suitable to the limited resources IoT.

Alarm and Notification System:

An alert mechanism is the sending of email notifications with the use of the Simple Mail transfer protocol (SMTP) in the event of an attack being detected. This keeps the users updated on the current trends in real time, and possible security threats can easily be addressed. To enhance the practical usability of the solution, the alert system enhances timely and actionable information.

All these tools and techniques as a whole are sufficient to make the system efficient, scalable, and able to handle real-time internet of things trafficking. A set of software

tools, machine learning techniques, and hardware are combined to produce a complete and usable instruct on IoT security.

Table 5: Tools and Technologies Used in the Project

Tool / Technology	Version / Type	Purpose
Python	3.x (Open-source)	Primary programming language
Pandas	Latest stable	Data manipulation and preprocessing
NumPy	Latest stable	Numerical computation
Scikit-learn	Latest stable	ML algorithms (LOF, RF, PCA, metrics)
Matplotlib / Seaborn	Latest stable	Data visualization and performance plots
Wireshark	Open-source GUI tool	Network traffic capture and analysis
Joblib	Scikit-learn utility	Serialization of preprocessing artifacts
ESP32	Espressif 32-bit MCU	IoT device generating real network traffic
Smart Bulb	Consumer IoT device	Secondary IoT endpoint in experimental setup
smtplib (Python)	Standard library module	Email alert delivery via SMTP

3.7 Constraints: Cost, Sustainability, and Feasibility

Every engineering project possesses a range of constraints which characterize the space of solutions and impact the choice of design. The IoT security system proposed has been made with the expectation of the constraints in terms of cost, sustainability and technical feasibility. The following are these limitations.

The issue of cost constraint was also taken care of under the influence of the strategic decision of open-source software along with the application of Python libraries that are free and can be used in all the software components. Hardware that will be used in the experiment, namely ESP32 microcontroller modules and a consumer-friendly smart bulb are inexpensive and readily available in consumer electronics retailers. None of the special software licences or special cloud computing tools were required and consequently the total project cost was well within the budget constraints of a student research project.

The extensible and modular system architecture was used to overcome the constraints of sustainability. Individual components can be retrained on new information, replaced by new and improved algorithms or new functionality added without requiring the redesign process of the entire system. The open-source tools are used in order to eliminate reliance on commercial software and ensure the system can be maintained as the software ecosystem evolves. Minimal computational footprint of the selected algorithms ensures energy-efficient execution of the algorithms in training and inference.

Feasibility issues were addressed with a well-organized staged implementation schedule which broke the project into small, achievable steps. This strategy made sure that the progress could be followed in a systematic way and that any slacking or technical difficulties of one phase could be detected and overcome without compromising the next stages. The technical risk of algorithm development and debugging was minimized by the choice of well-documented, and mature algorithms with existing Scikit-learn implementations.

3.8 Justification of Selected Approach

The choice of hybrid LOF and Random Forest approach is driven by a combination of theoretical, empirical, and practical factors that altogether prove the effectiveness of the approach compared to other strategies to address the particular situation with smart home IoT security.

Theoretically, the hybrid method explicitly resolves the inherent complement of unsupervised anomaly detection, and supervised classification. Supervised classification can be used to categorize threats accurately to those that are already

known, and anomaly detection can be used to detect deviating behavior that could be a new form of attack. Both methods are not complete in their coverage; they can be combined to achieve a certain degree of detectability that each can not achieve on its own.

As it has been proven empirically not only the findings of the Phase-I experimentation but also the literature review in general substantiate the decision to choose the LOF and the Random Forest as the specific algorithms of the hybrid framework. LOF performed well in the first experiments in terms of anomaly detection and was capable of identifying attack traffic patterns with high recall. Random Forest performed higher than other supervised algorithms that had both been tried on initial tests and are particularly robust in class imbalance and high dimensional feature space.

In practice, the resource resources and computational efficiency of the selected approach typically satisfy the constraints of the smart home IoT application. LOF and Random Forest inference are also computationally inexpensive as compared to deep learning alternatives, and can run in near real-time on customary computing resources. The available Scikit-learn applications reduce the risk of the development process and can test and be trustworthy with the behavior of the algorithms. The modular pipeline structure can withstand the future extension and modifications as the threat environment is evolving.

The systematic serialization of all objects of transformation and their uniform application in both training and inference phases is one of the major practical problems which are occasionally understated in published systems, and is where the preprocessing pipeline design is aimed. The design ensures the system is compatible with a large variety of data distributions and deployment configurations, an attribute that is essential to a reliable real-life implementation.

All in all, the hybrid solution identified is theoretically justifiable, experimentally validated, computationally viable and practically applicable, and provides a solution to the IoT security problems expressed in the problem statement. It offers a fair trade off between the coverage of the detection, classification, and computational efficiency and operational utility and this qualifies it to be well adapted to be used in the context of smart home IoT.

CHAPTER 4

DEVELOPMENT AND IMPLEMENTATION

4.1 Implementation Plan and Phase-II Progress

Accomplishment of Phase II was in a systematic manner. We had an action plan with objectives to ensure the project was achieved smoothly and successfully. The entire procedure was carried down into sections, such as preparing the data to lead to a model testing it and assembling all that. We separated it into sections so that we could do one at a time, doing things in a consistent manner to ensure that the entire system was working smoothly.

The initial procedure was to deal with the information on the internet traffic that was generated by tools such as ESP32 modules and smart bulbs. There was much information in this data about the behavior of the packets of data, when that occurred, the protocols involved and how devices were communicating with one another. We were required to tidy the data to improve and ensure it was all consistent. We eliminated those things that were trivial, or repeated. The missing information was fixed, too. We used Wireshark to capture the Internet of Things network traffic in time. This was done when the network was working normally and when it was, under attack.

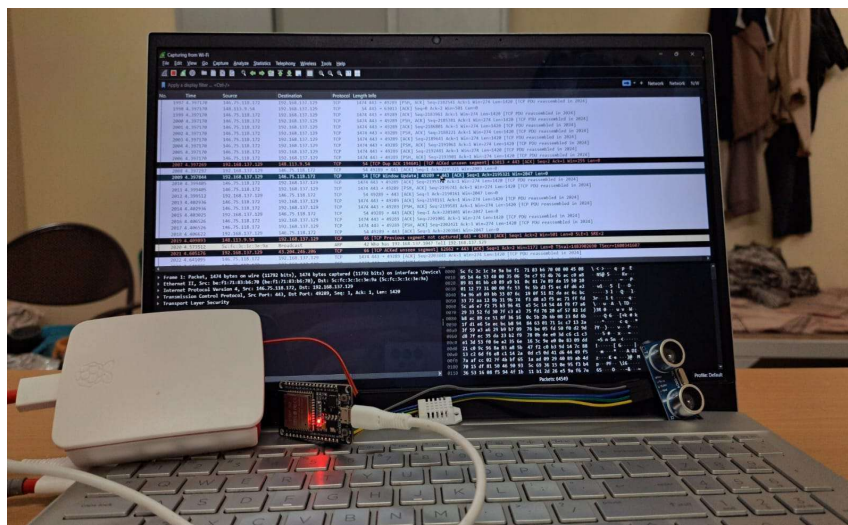


Fig. 2 : Real-Time Network Traffic Captured using Wireshark

The captured packets have details, like TCP/IP communication. They include packet sizes and timestamps. These are used for extracting features. The packets have protocol-level details.

Changed the categories into numbers so the computer could understand it. Then we made sure all the data was on the level. We also used a method called Principal Component Analysis to make the data simpler and get rid of information. This made it easier for the computer to process the data and saved time. The network traffic data from devices, like ESP32 modules and smart bulbs was the main thing we were working with.

The second thing was to come up with a model capable of discovering things not being common. This model involves the use of an approach named the Local Outlier Factor algorithm [10]. We also fed this model on internet traffic data, as it will learn what is typical and discover items that are anomalies. In this manner, the system is able to identify attacks that are new.

Then we made another model that can tell what kind of attack it is. We used the Random Forest algorithm for this [9]. We trained this model with data that has types of attacks so it can put the things that are not normal, into categories of attacks. This means the model can say what kind of attack the anomaly detection model found.

After the models were made the LOF and Random Forest models were checked to see how well they worked. This was done by looking at things like how accurate they were and their F1-score. The models were also improved to make them better, at finding the things they were looking for and to reduce the number of false positives. A lot of care was taken to make sure that the LOF and Random Forest models were treated the way when they were being trained and when they were being tested. This was done by saving the steps that were taken to get the data ready and using them again when the LOF and Random Forest models were being checked.

In the part we put everything together. This means we combined the parts that get the data ready the part that simplifies the data the part that finds things the part that figures out what things are and the part that sends warnings. We then tried out the system with data to see if it could really find problems as they happen and if it would work well in

different situations. We finished building the system on time. It works as planned. Now we have a security system, for internet connected devices that can find and identify network attacks quickly.

Table 6: Implementation Plan and Timeline

Phase	Activity	Description	Duration	Status
1	Data Collection & Preprocessing	IoT traffic capture, cleaning, encoding, scaling, PCA	Weeks 1-8	Completed
2	LOF Model Development	Training, parameter tuning, anomaly detection validation	Weeks 9-13	Completed
3	Random Forest Development	Training, class balancing, hyperparameter tuning, evaluation	Weeks 14-20	Completed
4	System Integration	Pipeline assembly, preprocessing consistency, end-to-end testing	Weeks 21-26	Completed
5	Alert System Development	SMTP email notification module implementation and testing	Week 27-29	Completed
6	Evaluation & Validation	Performance metric computation, confusion matrices, ROC curves	Weeks 30-32	Completed
7	Documentation	Report writing, figure preparation, reference compilation	Weeks 33-34	Completed

Table 7: Phase-II Milestone Progress Tracking

Milestone	Description	Target Date	Outcome
M1	Complete dataset collection with all attack types	Week 8	Achieved - Full dataset ready
M2	Preprocessing pipeline finalized and serialized	Week 8	Achieved - Artifacts saved
M3	LOF model trained and evaluated	Week 13	Achieved - 92.56% accuracy
M4	Random Forest model trained and optimized	Week 20	Achieved - 76.76% accuracy
M5	End-to-end pipeline integrated and tested	Week 26	Achieved - Pipeline functional
M6	Alert system implemented and validated	Week 29	Achieved - Email alerts working
M7	Full system evaluation completed	Week 32	Achieved - All metrics computed

4.2 Development / Execution of Major Components

It was a project that was done. Much valuable furniture was installed. Every component of the Internet of Things security system has something to do. We tore these parts apart in order to be easily changed and have the entire system working. The biggest components produced in this period are the Internet of Things security system components. The following are among the Internet of Things security system:

- Data Preprocessing Module:** The preprocessing module will be set up to clean and convert raw network traffic data to be used in training and testing. The module performs different transformations to improve the quality and standardization of data. It includes dropping repetitive, empty, and overlapping columns, and imputing the missed values in the data. It also encodes categorical

variables in order to convert them into numerical values, and scales the features to a normal value to sample the features to the same level. In addition, to eliminate redundant and correlated features and preserve important features, Principal Component Analysis (PCA) is used to reduce features. This aids in cutting off computational costs and enhances the performance of the model.

- **Anomaly Detection Module (LOF):** LOF algorithm was used as the basis of the anomaly detection system. Only normal traffic data were used to train to capture normal patterns of the IoT network communications. LOF identifies anomalies using the local density of the data points and the points which are quite different to their local neighbourhood [10]. This enables the system to be able to detect zero-day or invisible attacks without a need of attack data to train. The parameters were fine-tuned to control the system like the number of neighbors and the contamination factor to obtain higher accuracy and a reduced number of false alarms.
- **Attack Classification Module (Random Forest):** The classification model was trained using the Random Forest algorithm to recognize the type of anomaly in attacks. The algorithm was trained on a labeled dataset of attack types, and it is able to classify multi-class. Methods such as oversampling were employed to address the imbalance of classes in the dataset and provide equal chances to various attack classes. It also incorporated feature importance in the state of emphasizing crucial variables that are used in classification. The ensemble method of random Forest helps to improve the accuracy, eliminate overfitting and generate credible prediction even in the presence of noisy data.
- **Alert System:** This was a system designed to send out alerts on the detection of attacks. The system is implemented with the use of the Simple Mail transfer Protocol (SMTP) that delivers the email notifications to the user in case an anomaly or an attack is noticed. The alert can give details like the nature of the attack, level of confidence and status. The module enhances the practical uses of the system like response time, and reliability of the system.

These components were developed in such a way that every step of the system such as data preparation, and classification, and alert generation were treated well. Another contribution to the overall robustness and efficiency of the system was that because of

the modular implementation, it was very easy to integrate and test each component of the system.

4.3 Integration and Overall Functioning

The components of the system were perfectly incorporated into a processing pipeline to ensure maintenance of the data consistency, ease of operations, and improved performance. The purpose of integration was to provide seamless data flow at all levels, between the data input and output. This was achieved by connecting the components together whereby the output of a given process was incorporated into the other process to form an automated systematic system of detection.

The system functions by initially loading data on the network traffic into the preprocessing module. In this case, the data is normalized as in the case of the training process, the missing values are filled in, the categorical variables are encoded, features are scaled, and the dimensionality of the data decreased with the help of the Principal Component Analysis (PCA). This is important in order to make sure that the models have the same information as the one fed to them in the training phase, thereby improving the predictive accuracy and stability of the system as a whole.

After preprocessing, data is sent to the anomaly detection module, which is based on Local Outlier Factor (LOF) algorithm. The LOF model compares the samples of the data to the norm of the usual network functioning. In case the information is determined as normal, then it is allowed to pass by. However, when detected as an anomaly, it is stamped as suspicious and put through additional processing.

This anomaly is then inputted into the attack classification module which is used to classify the anomaly as an attack class using the Random Forest algorithm. This employs the trends in the features to determine the probable type of attack and gives it a confidence rating. The score gives the confidence level of the prediction and helps in making decisions.

The results of the classification can be used in deciding that the input is either an abnormal pattern or a type of attack. Once an attack is detected, the alarm system is triggered to inform the user about the type of attack that has been detected as well as its confidence. The analysis, monitoring and performance of the systems are also logged.

Such integrated workflow also enables the system to function in an organized, efficient and near real time manner. The combination of the preprocessing and anomaly detection and classification and alerting approach, maximises the detection accuracy reducing the false alarms and increases reliability of the security of the IoT network.

4.4 Challenges Encountered and Solutions

When we were working on the Internet of Things security system we had to deal with a lot of problems. The Internet of Things security system had issues, with handling data how well the model worked and if the system was consistent. We found ways to fix these problems with the Internet of Things security system so it would work well and be reliable:

Table 8: Challenges Encountered and Solutions Applied

Challenge	Root Cause	Solution Applied
Feature mismatch between training and testing	Column order and presence differed between datasets	Saved training feature list; enforced alignment at inference time
High dimensionality of network data	Wireshark exports 60+ features, many redundant	Applied PCA to retain 95% variance in fewer dimensions
Class imbalance in classification dataset	Normal traffic vastly outnumbered attack traffic	Applied Random Over-Sampler; used class weighting in RF
High false positive rate in LOF detection	Default LOF parameters too sensitive	Tuned n_neighbors and contamination via cross-validation
Pipeline inconsistency during deployment	Preprocessing parameters not saved after training	Serialized all artifacts (scaler, PCA, encoder) using joblib
Real-time data flow performance	Multiple pipeline stages creating processing latency	Vectorized operations; eliminated redundant computations

- Feature mismatch between training and testing data:** One big problem we noticed was that the feature columns did not match between the training and testing data. When the feature order was different or some attributes were missing during testing the model made predictions. This was fixed by

saving the feature structure from training and making sure the columns lined up correctly during detection. As a result the input data fit the format that the trained models were expecting. The feature columns had to match to get accurate predictions, from the model. The feature structure was crucial for the model to work properly. We made sure to use the feature columns for both training and testing.

- **High dimensionality of network data:** The dataset had features about network traffic. This made it hard for the computer to process and affected how well the model worked. To fix this we used Principal Component Analysis or PCA for short. PCA helped reduce the number of features. It kept the important information. This made the model work better and faster. It also helped prevent the model from being too specific, to the training data. The model efficiency improved. The processing time went down. Overfitting was minimized because of PCA. The model worked with network traffic features. These features were reduced in number. They still had key information.
- **Moderate classification performance:** The Random Forest classifier did okay at first. It had some issues. The problem was that some classes had a lot data than others and some features looked similar across different types of attacks. To fix this we used techniques like oversampling to make sure all classes had a number of examples. This helped a lot. We also adjusted the models settings to get the results. The goal was to make the Random Forest classifier better, at figuring out the attack types. By doing this we were able to improve its accuracy. The Random Forest classifier and data balancing techniques and hyperparameter tuning all helped improve the classification accuracy.
- **False positives in anomaly detection:** The model that finds unusual data using Local Outlier Factor or LOF had a problem. It marked data as unusual too often. This was wrong. We had to fix it. We adjusted the LOF settings, like how neighbors it checks and how much contamination it allows. We also changed the thresholds for detection. These changes helped the model get better at finding data without making too many mistakes. It now has a

balance, between catching unusual data and being accurate. The LOF model is now more reliable.

- **Pipeline inconsistency during deployment:** The training and deployment phases had to be consistent. When we tested the system the steps we took to get the data ready were different. That made the predictions not very reliable. We fixed this problem by saving all the things we did to get the data ready like filling in missing values scaling and using PCA transformation models. We reused these things when we tested and deployed the system. This made sure that the data was processed in the way and it made the system more stable. The training and deployment phases were consistent because we reused the preprocessing components, including imputation, scaling and PCA transformation models during testing and deployment which was a part of the data processing.
- **Handling Real-Time Data Flow:** The system had to handle data quickly. This was a challenge because the data had to go through stages. We made each part of the system work better and got rid of work it was doing. This made the system work faster. It could find attacks on time.

These challenges taught us a lot about designing a system. We used what we learned to make our solution stronger, more accurate and more reliable. The solutions we put in place made sure the system works well in situations. This makes it a good choice for security, in Internet of Things applications.

4.5 Observations

During the process of setting up the system we had observed some crucial things about how their data was being processed as well as how the system was operating. The section that we prepare the data is actually crucial in ensuring that the models are accurate and provide results. When we manage to deal with the missing information properly and the categories are marked with right and the features are, on the scale, the data would appear much more beautiful. We also performed a method called Principal Component Analysis, which is one method of reducing the data size making the data easier to handle. This assisted in eliminating part of the noise besides making it cheaper to operate and functions of the models more efficient.

Local Outlier Factor model is very effective in detecting things that do not appear right in the network. It is also capable of discovering attack patterns that are new to us which can be quite handy to Internet of Things devices where new threats are constantly emerging [10]. This is quite good using the Local Outlier Factor model.

Local Outlier Factor model is effective in choosing the settings. Unless we choose the settings, in the case of the Local Outlier Factor model it may warn us falsely. We must, therefore be cautious when constructing the Local Outlier Factor model in order to ensure that it functions properly and identifies the threats. The Random Forest classifier was successful in making its guess about which type of attack was occurring.. In some cases it took a while to figure it out since at times attacks resemble most closely within the network. This complicated differentiating them by the Random Forest classifier [9]. It was a couple of missteps. With this issue the Random Forest classifier managed to discover a handful of significant relationships, in the data and decent guesses most of the times. We adjusted the settings so that the Random Forest classifier would improve a lot in classifying the attacks when we ensured that the classifier was considering the amount of each kind of attacks.

The hybrid method is quite excellent since it involves the use of both methodologies in anomaly detection and classification. This is a combination that is more effective, than single model. Anomaly detection section examines the data. Discovers the perplexing material. Then the classification section determines the type of attack it is. This will assist in locating the attacks precisely and lower the occurrence of false alarms. Anomaly detection along with classification will help strengthen the system and make it more proven [14][28]. To achieve higher results, one should go through the hybrid solution since it employs both anomaly detection and classification.

Also it was observed that whether the model is effective or not actually depends on the quality of the data that features it has and the way the data is formatted. Tiny differences in what features are used and how the data is prepared can result in substantial changes in the final results. It was quite an effective system to absorb data and handle it as desired each time and provide results in a transparent format. The alert system was

more useful in real-life scenarios as it only sent notifications in cases where the alert system detected an attack.

When we introduced this system it was working fine. What we observed indicates that our technique works well to detect and categorize cyberattacks in houses with Internet of Things technology. Internet of Things technology has the capability to attack the cyber. Put under this technique. This predisposes it to be useful in the world provided we engage in some further improvements in the system, to be able to detect cyberattacks as far as the Internet of Things technology is concerned.

CHAPTER 5

RESULTS, ANALYSIS AND VALIDATION

5.1 Testing / Evaluation Methodology

To determine the effectiveness of the Internet of Things security system, it was tested. To ensure that the results are fair, we applied a method of testing. We were interested to know whether the system is able to identify problems and inform what types of problems they are. Conditions in the world were tested under the system to determine whether it is effective. We sampled the network data. Cleaned it up. We ensured that the data was fine and fit to work. The reason we did this so is that the system can learn based on the data and make decisions as it is in use. This data was tested in the Internet of Things security system to determine how well the system can identify issues and informs about the type of issues they represent.

The data was divided into two, training part, and testing part to help us understand the extent to which the model performs well. The Local Outlier Factor model was used to detect anomalies. We have only trained it with normal internet traffic data, in order to be able to learn what is normal. In this manner the model will be able to discover patterns that are not normal during the testing of the model. To find out whether or not Local Outlier Factor model can distinguish between normal and abnormal attack traffic data, we tested it with a dataset that was a mixture of attack traffic data. We examined things such as how accurate it is how precise it is how well it recalls things and its F1-score in order to see how well it can detect anomalies. Positives and false negatives helped us get a better conception of the performance of the Local Outlier Factor model.

To classify attacks the Random Forest model was trained on some labeled data containing the types of attack. A set of data was then tested with this model to determine how well it is able to distinguish between different types of attacks. To examine the effectiveness of the model we considered such parameters as the way it is correct, the accuracy of memory and its F1-score. We also created confusion matrices in order to create a glimpse of each kind of attack that assists us in observing when the model hits its mark and when it errs with various kinds of attacks. Classification of the attack matters. Random Forest model is applied in classifying attacks.

To test the performance of the classification model when there are classes we also tested the Top-3 accuracy. This value validates whether the right attack type exists in the three categories that the model makes predictions. It allows us to know the extent to which the model learns, although it may not always make the prediction with the class. The model is particularly useful when there are numerous types of attacks with network characteristics. Top-3 accuracy gives feedback, regarding the performance of the model. This metric is effective in the classification model to deal with -class scenarios. With this supplementary evaluation measure, one obtains an insight into the model.

In general our assessment approach tests the system in regards to detection and classification. Performance measures and analysis techniques are utilised by us. This assists us in knowing the degree to which our model can be accurate, reliable and useful under the network security circumstances. The tests functionality presents an image of the model functionality. It also demonstrates whether the model is applicable to the life internet of things network security settings. The system analysis is all inclusive of detection and classification. It guarantees us that we know the accuracy and reliability of the models. The testing assists in the use of IoT-based network security.

5.2 Results Obtained

The Internet of Things security system that they developed was evaluated in two ways: finding things that are not normal and sorting things into lots of groups. Internet of Things security system outcomes demonstrate that such an approach to things is effective. They considered the functionality of individual components of the Internet of things security system with the standard measures of the effectiveness of the functionality. Overall, the findings, in the case of Internet of Things security system, are as follows:

5.2.1 Local Outlier Factor (anomaly Detection Performance)

LOF model fared fairly well with an accuracy of approximately 92.56. This indicates that it can be used to identify normal and abnormal network traffic. The model is also highly accurate at 96.89 that is, it does not give false alarms. Most of the things it qualifies as weird are abnormal. Its recall rate is 93.33% which is quite high. This implies

that most of the anomalies are picked up by the LOF model. The F1-score is 0.95. This score represents a trade off between recall and precision. The results of the performance are presented in Table 5.1 below..

Table 9: Anomaly Detection Performance Metrics (Local Outlier Factor)

Metric	Value	Interpretation
Accuracy	92.56%	Correctly classified 92.56% of all test samples
Precision	96.89%	96.89% of flagged anomalies were true attacks
Recall	93.33%	93.33% of actual attacks were correctly flagged
F1-Score	0.95	Strong balance between precision and recall
AUC-ROC	0.96	Excellent discriminative capability

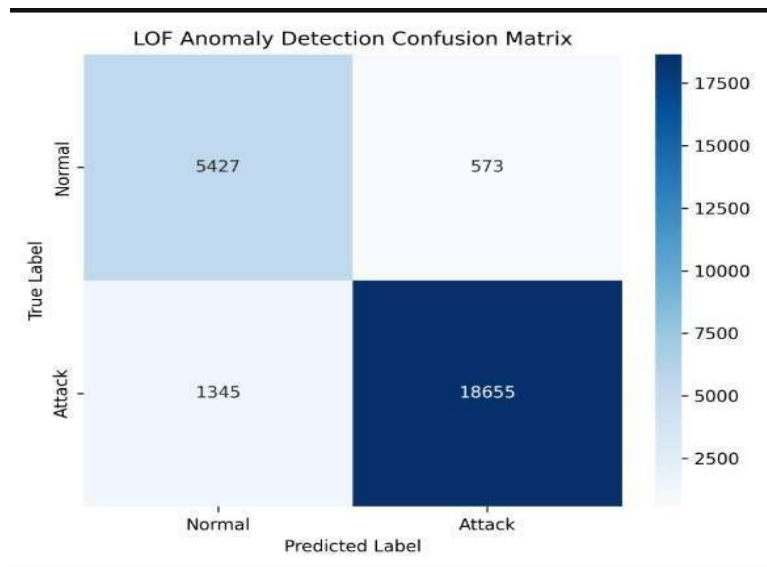


Fig. 3 : LOF Anomaly Detection Confusion Matrix

Figure 5.1 is a confusion matrix as it tells us that the LOF model was able to make most of the samples of attack traffic correct. When it claimed normal traffic was bad it did not make errors. This implies that the LOF model is proficient in aware of what represents typical traffic. Attack traffic samples were also not not missed by the LOF model. This indicates that the LOF model performs well, in tracking down the traffic of attacks of any type. The LOF model was able to employ the test data.

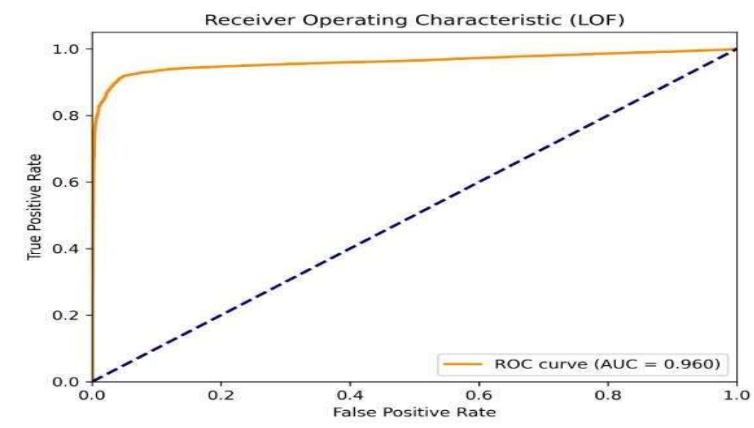


Fig. 4 : ROC Curve for LOF Anomaly Detection (AUC = 0.96)

Figure 5.2 gives us the ROC curve of how the positive rate and the false positive rate vary with changes in the classification threshold. The AUC of the LOF model is 0.96. This implies that the LOF model is very effective in narrating. It performs better than a random classifier. LOF model can, therefore, be highly applicable in identifying anomaly in IoT applications. The LOF model can differentiate between things, enabling it to be highly useful, in the anomaly detection applications of IoT.

5.2.2 Attack Classification (Random Forest)

It was the Random Forest attack classification model that made some effort to determine the type of attack that was occurring. It could put the bulk of what it found into the correct category. It was doing well, as you can see in Table 5.2. The model of the Random Forest attack classification was fairly accurate in distinguishing the types of attack.

Table 10: Attack Classification Performance Metrics (Random Forest)

Metric	Value	Interpretation
Overall Accuracy	76.76%	Correctly classified 76.76% of all attack test samples
Macro F1-Score	0.76	Balanced performance across all attack classes
Weighted Precision	~0.78	Most flagged attacks correctly identified
Weighted Recall	~0.77	Adequate coverage across attack categories
Top-3 Accuracy	~94%	Correct class in top-3 predictions in majority of cases



Fig. 5 : Random Forest Classification Confusion Matrix

Figure 5.3's confusion matrix shows that the Random Forest model gets most of the attacks right. There is some confusion between types of attacks that have network features, especially between some DDoS and DoS types. Other studies have also found this to be true. It's hard to tell the difference between network-layer attacks with

different amounts of traffic and packet rates just by looking at network features. The Random Forest model and DDoS and DoS attacks are good examples of this. The results from the confusion matrix and the Random Forest model make this very clear. We see that DDoS and DoS variants are often put in the wrong category. The network parts of DDoS and DoS attacks are very similar. This makes it hard to put them in the right category. The Random Forest model works most of the time for attacks.

```

SECURITY ALERT
Prediction: Known Attack
Attack Family: ManInTheMiddle

SECURITY ALERT
Prediction: Known Attack
Attack Family: Network

SECURITY ALERT
Prediction: Known Attack
Attack Family: Reconnaissance

```

Fig. 6: Detection Output - Attack Classification Result

```

NORMAL TRAFFIC DETECTED
Prediction: Normal

```

Fig. 7: Detection Output - Normal Traffic Result

```

SECURITY ALERT
Prediction: Abnormal Pattern Detected

```

Fig. 8: Detection Output - Abnormal Pattern Detected

The overall results show that the LOF-based anomaly detection part works very well, with a high accuracy rate of about 92.56% and strong precision and recall values. This makes it very reliable for finding unusual network behavior in IoT settings.

5.3 Analysis and Interpretation of Results

The test results show that our proposed IoT security system works well. It performs great in its two parts. The LOF anomaly detection model is really good at detecting anomalies. It has precision, recall and AUC-ROC values. These values are as good as or even better than similar systems. These systems were reported in studies that looked at IoT traffic detection tasks. The system's high precision of 96.89% is especially important. This is because it means the system rarely sends out alerts. This is crucial for it to be accepted in real-world use. The system is reliable, for use.

The Random Forest classification model does a good job and gets it right about 76.76 percent of the time when it looks at all the different kinds of attacks. This is not as good as it is at finding anomalies. It is still good for a problem where we have to figure out which kind of attack is happening and there are a lot of them that look similar. The Random Forest classification model is really good at narrowing it down to the three possibilities it gets this right about 94 percent of the time. This means that when the Random Forest classification model does not pick the right attack it usually has the right one in its top three guesses. This tells us that the Random Forest classification model has learned to recognize what is important, about each kind of attack the ones that are hard to tell apart from the others.

The study shows that the kind of attack and how well we can tell it apart is connected to how good we're at figuring out what is going on. When we look at attacks like DoS attacks that make a lot of packets or MITM attacks that change the way traffic is routed we can tell what they are easily. We can classify these attacks most of the time. On the hand some attacks like brute force attacks are harder to spot because they do not always look the same. This means we do not do well at classifying these attacks. This tells us that we need to work on making our features better so we can get better at classifying attacks in the future.

The dimensionality reduction, as well as the preprocessing, stages of the pipeline played an essential role in achieving the performance results obtained. The dimensionality reduction stage with the help of PCA contributed to reducing the number of features. This was achieved by moving from more than 60 original features to principal components. It was beneficial in speeding up the learning process, as well as increasing the efficiency of the model. We paid special attention to preprocessing all datasets identically. In this way, it was guaranteed that the learning data and testing data were processed in the same manner. It is crucial because our findings showed that in case of different pre-processing steps the results were poor.

There are several advantages associated with the use of the hybrid model compared to that of using each of the components separately. The LOF model had a rate of false positives, but the Random Forest classification module managed to correct this issue by filtering anomalies that had unclear characteristics. They did not fit in known attack patterns. The ability of LOF to identify patterns in traffic without the use of a labeled dataset distinguishes it from the other models. The combination of both LOF and the Random Forest makes the algorithm more robust compared to the use of either separately.

5.4 Validation of Objectives

This part shows that we have done what we said we would do in Chapter 1. We have completed all the project objectives. The project objectives have been met by putting the proposed system in place and seeing how well it works. The proposed system has been a success. The project objectives stated in Chapter 1 are now complete.

The main goal was to create a smart home with internet connected devices. We did this by setting up ESP32 microcontrollers and a smart bulb device in a network. These devices sent internet of things traffic that we used for all our other work. The smart home with internet connected devices was the thing we were trying to make. We used the ESP32 microcontrollers and the smart bulb device to make it happen in a network that we controlled. The traffic from the home with internet connected devices was very important, for what we did next.

The second thing we wanted to do was to capture and look at the network traffic. We used Wireshark to record what is happening with the traffic, at the packet level when everything is working normally. We then looked at the data we captured to see how Internet of Things devices normally communicate with each other. This helped us understand the properties of normal Internet of Things device communications.

The third thing we wanted to do was to try out kinds of attacks and collect data on the traffic from these attacks. We did this by running simulations of Denial of Service attacks Distributed Denial of Service attacks, brute force attacks and Man in the attacks on our Internet of Things experimental environment. Then we used Wireshark to capture the traffic data from these Internet of Things attack simulations. We got the data we needed from the Denial of Service attack simulations the Distributed Denial of Service attack simulations the brute force attack simulations and the Man, in the Middle attack simulations.

The fourth thing we wanted to do was to get the collected data ready and look at it. We did this by making a plan to clean up the data fill in the missing parts change the way the features were shown make everything the same size and use something called PCA to make the data smaller. We did all these things to the collected datasets.

The fifth and sixth things we wanted to do were to make a system that uses machine learning to find intruders and to make this system able to respond to threats. We did this by making the LOF anomaly detection model, the Random Forest classification model and the email alert system that uses SMTP. When we put all these things together the system works well and can find and classify threats, to IoT networks very quickly.

A screenshot of a terminal window with a black background and green text. The text reads: "SECURITY ALERT", "Prediction: Known Attack", "Attack Family: Reconnaissance", and "Summary email alert sent!".

```
SECURITY ALERT
Prediction: Known Attack
Attack Family: Reconnaissance
Summary email alert sent!
```

Fig. 9: Email Alert Notification for Detected Attack

Overall, the proposed system fulfills all defined project objectives. It can detect anomalies, classify attacks, generate alerts in time and work well with other systems. The results show that our solution is good at making IoT networks more secure. This makes it suitable for use in homes. The system meets all the objectives we set out to achieve. It is effective. Can be used in real-life situations. The developed solution enhances network security. It is suitable for deployment in smart home environments, with IoT network security.

5.5 Comparison with Expected / Existing Outcomes

The proposed IoT security system was. Its performance matched with what we expected. We also compared it with security systems that are already out there for detecting intrusions in IoT. Traditional security systems [32][13] like those that detect attacks based on signatures have a major flaw. They can only spot attacks that they already know about. This means they are not very good at dealing with changing threats. Our system on the hand is more flexible and adaptable. This is because it uses anomaly detection.

With anomaly detection we can identify both unknown attacks. We used the Local Outlier Factor model. It performed well. It gave us accuracy and a good balance between precision and recall. This matches what we expected. Is similar to what other studies on IoT security have found. The results are good for security and the proposed system. The Local Outlier Factor model is good, for the proposed IoT security system. The Random Forest model works well even though it is not as good as some deep learning models.

The comparison shows that the proposed hybrid system does really well. It is as good as systems that are already out there. The hybrid system can detect anomalies accurately it gets it right 92.56 percent of the time. It also does well on a test called AUC-ROC it scores 0.96. This is better than some systems that use something called LOF. Some systems that use learning can be more accurate but they need a lot of power to work. They are not good for use in homes because they need too much power. The hybrid system is better for homes because it does not need as much power. The Random Forest part of the system is 76.76 percent accurate. This is the same as other systems that use something called supervised ML. These systems are used to stop attacks on the internet in homes. The hybrid system is good, at stopping these attacks.

Deep learning models can be more accurate because they find patterns but they need a lot of data, powerful computers and take a long time to train. The Random Forest model is a choice because it is accurate does not use too much power and it is easy to understand how it makes decisions. This makes it suitable for environments where resources are limited. The performance of the Random Forest model is as expected for problems with classes where attacks have similar characteristics. The Random Forest model is effective for use. It provides a balance between accuracy, power used and ease of understanding. Random Forest model is more suitable, than learning models in this case.

The proposed system has an advantage. It uses an approach that combines two things: finding things that are not normal and putting things into categories. Most systems only do one of these things.. This system does both. It has a framework that puts these two components together. This makes the system better at finding things. It first finds behavior that's not normal. Then it puts that behavior into a category of attack. The system works overall. It has false positives. It is more reliable, than systems that only use one model. The proposed system is really good because it uses this approach.

The suggested solution has an edge compared to previous solutions. It has everything integrated, from beginning to end. Existing solutions have mostly concentrated on particular aspects such as identification and classification. They lack a complete process from preparation to alert generation. The proposed solution covers all stages in a single efficient and user-friendly system. It is applicable in households with a significant number of IoT devices. The suggested system is appropriate for world-wide smart home IoT implementation since it is efficient.

In summary, our methodology proves to be an effective means of pattern recognition, which provides accurate classification results. The methodology also comes with certain strengths. The strengths of the methodology include its compatibility with current systems, real-time operations, and practicality. The added strengths make it evident that our methodology represents an all-in-one solution for securing IoT networks. The proposed methodology meets anticipated requirements for performance

in anomaly detection. It also provides competitive results in classification.

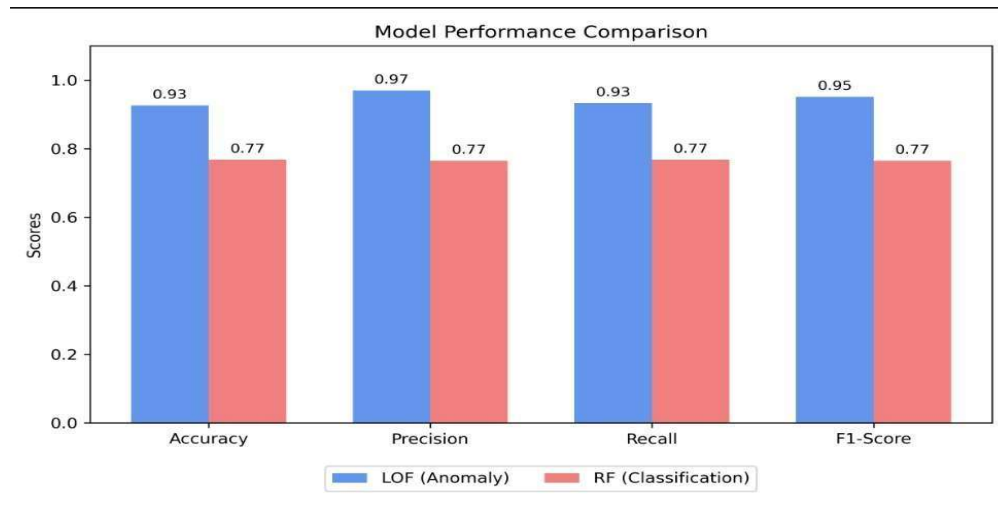


Fig. 10: Overall Model Performance Comparison Graph

```

--- LOF (Anomaly Detection) ---
Total Samples Tested: 302
Normal Traffic Correctly Passed: 133 out of 150 (88.67%)
Attacks Successfully Detected: 146 out of 152 (96.05%)
Overall Accuracy: 92.38%

--- Random Forest (Family Classification) ---
Successfully Classified Families: 98 out of 146 attacks detected (67.12%)
    
```

Fig. 11: Overall Model Performance output

CHAPTER 6

CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

Indeed, the design, development, implementation, and evaluation of the proposed machine learning-based smart home IoT security system have been achieved and successfully conducted. Such smart home IoT security system is able to detect, classify, and alarm about any cyberattacks on the network in nearly real time. This system is indeed capable of tackling and addressing the major shortcomings of the existing security systems. This hybrid approach of detecting and preventing the attacks is indeed a unique combination that provides a comprehensive and efficient solution to the problems of security systems. This smart home IoT security system is indeed an improvement due to the inclusion of unsupervised and supervised approaches.

From the test results, it can be concluded that the Local Outlier Factor algorithm performs excellently. It shows accuracy, which stands at 92.56 percent. The precision is 96.89 percent. Recall is at 93.33 percent. F1-Score measures 0.95. AUC-ROC Score is 0.96. These values prove that the Local Outlier Factor algorithm has the ability to detect IoT network behavior. This behavior includes traffic flow from the new attacks. The very high value of precision indicates that the algorithm has a false positive rate. It is crucial for practical use since it prevents the system from creating false alarms for operators. The Local Outlier Factor algorithm effectively detects deviations from IoT network behavior. It detects traffic flows.

The Random Forest attack classification model accuracy rate is 76.76%. The model performs effectively in classifying the type of attacks that occur. The model has been trained to identify many types of attacks and differentiate them from each other. This assists the persons responsible for managing these attacks in making decisions.

The Random Forest attack classification model is effective in distinguishing the three categories of attacks. The accuracy rate of the model for identifying the categories is 94 percent. This indicates that the model successfully identifies the unique characteristics of these three categories of attacks.

The Random Forest attack classification model can differentiate attacks despite their similarity. The hybrid LOF-plus-Random Forest system is really good. It does a job than

using LOF or Random Forest on its own. The hybrid LOF-plus-Random Forest system sends out false alarms. At the time it still catches all kinds of attacks whether they are known or unknown. The way it works is that it uses a two-step process. First LOF finds things that're unusual. Then Random Forest takes a look at those unusual things. This helps get rid of alerts. The hybrid LOF-plus-Random Forest system is good, at finding attacks without sending out too many false alarms. The LOF-plus-Random Forest system is very effective.

How we preprocessed our data was critical for our algorithm to work effectively. The consistency of how we processed the data allowed us to have the same results from the algorithm each time we ran it. We used Principal Component Analysis (PCA) on our data set to lessen the number of dimensions. This increased performance, as well as lessened the chances of the algorithm performing poorly due to the curse of dimensionality. Another key feature in our preprocessing process was that we had examples of all attack types in the dataset for our algorithm to learn. This allowed our random forest classifier to properly classify both majority and minority attack classes.

Overall, the email alert system performed effectively throughout the test process. Notifications were sent out in a timely manner whenever an issue occurred. Notification messages included critical information such as type of attack and date of occurrence. The email alert system is beneficial since users can be alerted to any security breaches and take necessary action without having to monitor the network continuously. The email alert system is important in making the entire system more efficient for users.

In conclusion, the proposed solution presents an approach to ensure that IoT networks in smart homes are protected against cyber-attacks. The system is easy to use. Minimal computing resources are required for its operation. Machine learning algorithms are used to detect behaviors and categorize attacks. This is a practical application in protecting IoT networks from malicious attacks. The proposed solution is just the first step in improving cybersecurity in home IoT networks.

6.2 Limitations of the Work

The system works perfectly when it is applied in accordance with its purpose. There are certain limitations that may affect the operation of the system in certain conditions. However, being open about these limitations will enable us to understand what is currently possible with the system and what needs to be done for its further improvement.

The main limitation concerns testing conditions of the system because it has been tested only in an experimental setting where several devices have been employed, for example, an ESP32 module and a smart light bulb in a laboratory network. At the same time, in reality, people have many smart home devices created by different manufacturers and functioning differently depending on their software. Thus, we do not know how the system would perform in these complex conditions.. We do not know how well the system will work in these complicated situations and it might be different from what we found out. The system was tested in an environment but real-world smart home environments are much more complex and the systems performance in these environments is still unknown which means the results we have might not be the same as what would happen in a real smart home with lots of different devices, like the system we are talking about the system and its ability to work well in these situations.

The dataset we used to train and evaluate the system is from hardware but it is not very big and does not have a lot of different types of attacks. We tested the system with the common types of attacks that smart home IoT devices face but we did not test it with less common or very complicated attacks like firmware injection or side-channel attacks or advanced persistent threats. We do not know if the system will work well with these types of attacks because we did not test them with the IoT dataset. The IoT dataset is. This is a problem. The system may not work well with types of attacks, on the IoT devices.

The LOF anomaly detection model is really sensitive to changes in the way the network normally works over time. When things like device firmware updates or changes, in how people use the network happen or when new devices are added the normal traffic pattern can change a lot. This can cause the LOF anomaly detection model to have false positives until the normal traffic pattern is updated. The thing is the current system does

not have a way to automatically update the traffic pattern. This means the LOF anomaly detection model can keep having positives until someone updates the normal traffic pattern.

On the other hand, Random Forest classifier does not perform adequately when classifying attack types having similar network features. This can be evident based on the outcome presented by the confusion matrix. It is challenging to differentiate among attack types having similar characteristics such as diverse forms of DDoS attacks solely based on network features. In an attempt to handle this challenge, one may consider incorporating temporal-based features or employing more advanced techniques that integrate various methods. This is necessary since the Random Forest classifier cannot classify attack types based on network features.

The system can find problems. Send alerts but it does not automatically stop the threats or take action. When we use the system for real it would be great if it could work with the network to automatically block traffic keep compromised devices away, from the rest or do something else to defend against threats when it finds them. The system can detect threats and the system should be able to work with the network to stop these threats. This has not been done in the system we have now.

Finally, the system has not been tested to see if it can handle a lot of devices and a lot of traffic at the time. The system is made in a way that should make it easy to scale up to Internet of Things networks. The algorithms used are also very efficient. However the Internet of Things system needs to be tested some more to make sure it can handle a lot of traffic from devices. This is especially important, for companies that have a lot of devices on their Internet of Things network. More testing is needed to make sure the Internet of Things system can handle all the traffic from these devices.

6.3 Future Scope

The proposed system is a starting point. It can be improved in ways. Here are some areas that can be developed further. One area is extending its capabilities. Another area is enhancing its performance. These areas have a lot of potential for growth.

We need to work on real-time streaming data processing. Now we are using traffic data from files that do not change. This is not the same as using network traffic. If we use libraries like Scapy or PyShark to capture packets and combine them with real-time streaming data processing frameworks like Apache Kafka or Apache Flink we can process live network traffic as it happens. This will make our system much better for use. Real-time streaming data processing is very important, for work. We can make our system work with real-time streaming data processing, which's a big deal. Real-time streaming data processing will help us a lot.

The adaptive model retraining mechanisms will help with the problem of the baseline drift over time. We can use learning algorithms or we can retrain the system offline from time to time with the traffic data we have collected. This will help the system get used to the changes in the Internet of Things network behavior. These changes can happen because of updates to the devices or changes in how people use the devices or changes, in the network configuration. If we have these mechanisms in place the Internet of Things network will be able to detect things even after it has been running for a long time. The adaptive model retraining mechanisms will make sure the system stays accurate without needing someone to update the model manually.

This may further improve our classification by analyzing the traffic in terms of statistical characteristics and quantifying them. For instance, this method may allow one to determine when certain packets arrive and how they are clustered in time, which could be helpful in differentiating between attacks that are too alike for a regular classification algorithm. Another potential avenue to explore would be to utilize ensemble learning approaches and specific feature selection techniques for traffic analysis.

The extension of the environment to more IoT devices will greatly increase its applicability. Indeed, incorporating devices from a range of makers such as smart speakers, sensors, cameras, and smart appliances into the training set would allow building a model that is more representative of typical households. Moreover, this will allow for testing in the conditions resembling real-world scenarios. The larger number of devices will allow for applying the proposed solution in various cases and will ensure greater reliability.

We could simplify its usage if we provided alerts and improved the dashboard. We should create an online interface through which we will observe what is currently occurring within the network traffic. We must also see whether we face any current threat. In addition, the monitoring should include historical data for more accurate insight into potential dangers. We could thereby significantly increase the usability of the system, especially among individuals without sufficient technical knowledge. Besides, it would help if we created notifications that would provide us with information regarding the state of the system whenever we are away from our computers. The system should be capable of recording all activities and notifying us about any errors. That way, we would significantly improve its usefulness in our workplace, particularly in regard to both the system and network traffic.

Another long-term solution we should implement in the system includes automated threat response capabilities. We could integrate them with existing network management systems or firewall rules engines in order for them to respond to potential dangers automatically. Such solutions would allow us to eliminate any threat before it occurs. The automation process requires the use of IoT gateway control interfaces.

The suggested approach would be a great fit for the Internet of Things network security. It enables the industry's further development. In particular, there are several methods through which one can enhance the system. They all are feasible to be implemented into the framework. Implementing these methods would enable the gradual improvement of the system, which, in turn, would ensure its high efficiency when applied to homes filled with smart devices. Thus, this system could positively impact the Internet of Things network security.